The Foundation - System Mission, Ubuntu, Terminal, and GitHub

Dr. TNOURJI, Abdellah
Research Scientist
The UM6P Vanguard Center

Autonomous Drone Engineering Internship

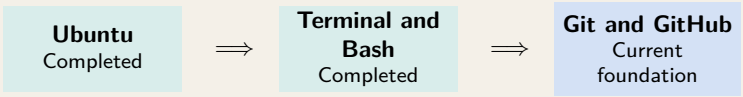
Mohammed VI Polytechnic University, UM6P , September 8, 2026

Outline

1 Git and GitHub Foundations

From Bash to Version Control

The terminal operates files; Git manages their engineering history



New Engineering Question

How can several students modify the same robotics project while preserving its history, identifying each contribution, and keeping the GitHub repository reproducible?

Central Principle

Git is not a replacement for careful engineering. It makes careful work visible, traceable, reviewable, and recoverable.

Why Robotics Needs Version Control

Robotics systems evolve through many connected files and contributors

Without Version Control

- files become `final`, `final2`, and `final_fixed`;
- working configurations are overwritten;
- changes cannot be attributed or reviewed;
- two contributors may replace each other's work.

With Version Control

- each meaningful change becomes a recorded checkpoint;
- experiments can be isolated on branches;
- collaborators synchronize controlled changes;
- earlier states remain available for investigation.

Project Connection

A change to a ROS 2 node may depend on a launch file, model, environment variable, or README instruction. Version control preserves these relationships as the system evolves.

What Is Version Control?

A structured history of a project's meaningful states

Working files $\longrightarrow$ Selected changes $\longrightarrow$ Recorded checkpoint $\longrightarrow$ Shared history

Intuitive Definition

Version control records how a collection of files changes over time. It helps engineers answer: *What changed? Who changed it? Why was it changed? When was it changed?*

Git and GitHub

A local version-control tool and an online collaboration platform

Git

- distributed version-control software;
- runs on the local computer;
- records commits, branches, and merges;
- can work locally without GitHub.

GitHub

- online hosting and collaboration platform;
- stores shared Git repositories;
- provides browsing, issues, reviews, and documentation;
- connects contributors through remote repositories.

Git and GitHub

Git: The Local Camera

Think of Git as a digital camera for your project.

- When you write a good piece of code, you take a "snapshot" (**commit**).
- If your next experiment breaks the drone, you can simply rewind to an older snapshot.
- **Limitation:** These snapshots live *only* on your physical laptop. If the laptop breaks, the code is gone.

GitHub: The Shared Gallery

Think of GitHub as an online photo album.

- You upload your local snapshots to the internet (**push**).
- Your 9 teammates can browse your code and download it to their own laptops (**pull**).
- **Advantage:** It acts as the ultimate backup. It is the "single source of truth" for the team.

Git and GitHub

Git: The Local Camera

Think of Git as a digital camera for your project.

- When you write a good piece of code, you take a "snapshot" (**commit**).
- If your next experiment breaks the drone, you can simply rewind to an older snapshot.
- **Limitation:** These snapshots live *only* on your physical laptop. If the laptop breaks, the code is gone.

GitHub: The Shared Gallery

Think of GitHub as an online photo album.

- You upload your local snapshots to the internet (**push**).
- Your 9 teammates can browse your code and download it to their own laptops (**pull**).
- **Advantage:** It acts as the ultimate backup. It is the "single source of truth" for the team.

The Daily Project Workflow

1. Write Code $\longrightarrow$ **2. Snap Photo** (`git commit`) $\longrightarrow$ **3. Upload** (`git push`)

Essential Git Vocabulary

A project-centered mental model before using commands

Repository	Project files together with the Git history and metadata.
Commit	A recorded snapshot of selected changes with author, time, and message.
Branch	A movable line of development used to isolate related work.
Remote	A named connection to another repository, commonly `origin` on GitHub.
Clone	A local copy of a repository, including its history and remote connection.
Pull	Retrieve remote changes and integrate them into the current local branch.
Push	Publish local commits to a remote branch.
Merge	Combine histories from different lines of development.
`.gitignore`	Rules describing untracked files that Git should normally ignore.

Before Cloning: Verify the Destination

Clone the repository once and into the intended parent directory

Pre-Clone Checks

```
cd ~
pwd
ls -la
which git
git --version
```

Question Before Execution

Will `git clone` create `quard_um6p_intership` inside the current directory, or does a folder with that name already exist?

Common Mistake

Do not clone the repository repeatedly inside itself or inside another accidental clone. If the target folder already exists, inspect it before taking action.

Access the Internship Repository on GitHub

Understand the platform before copying commands

Repository Address

`https://github.com/abdtnourji/quard_um6p_intership`

Inspect on GitHub

- repository name and owner;
- default branch;
- README and documentation;
- folders and recent commits;
- access permissions.

Obtain the URL

Select **Code**, choose the documented authentication method, and copy the repository URL exactly. HTTPS is the simplest starting point when no SSH key workflow has been configured.

Clone the Internship Repository

Create one local working copy with history and remote configuration

Purpose and Syntax

```
git clone <repository-url>
```

Internship Command

```
cd ~
git clone https://github.com/abdtnourji/quard_um6p_intership.git
cd quard_um6p_intership
```

What the Command Creates

The project directory, working files, commit history, branches known at clone time, and a remote normally named `origin`.

Verify That the Clone Is Correct

A successful-looking download still requires engineering checks

Verification Sequence

```
cd ~/quard_um6p_intership
pwd
ls -la
git status
git branch
git remote -v
git log --oneline -5
```

Expected Evidence

The path identifies the intended repository, `.git` exists, Git recognizes a branch, the working tree is initially clean, `origin` points to the expected GitHub URL, and recent commits are visible.

Explore Before Modifying

Understand the project structure and follow its documentation

Filesystem View

```
ls -la
find . -maxdepth 2 -type d | sort
find . -name "README*"
find . -name "package.xml"
```

Git-Aware View

```
git status
git branch
git log --oneline --graph -8
git remote -v
```

Professional Rule

The README is part of the engineering interface. Read setup instructions, folder responsibilities, prerequisites, and known limitations before executing the pipeline.

Use `git status` as the Dashboard

Ask Git what it sees before and after every important action

Purpose and Syntax

```
git status
git status --short
```

Interpret the Categories

- **untracked**: Git has not been asked to include the file;
- **modified**: tracked content differs from the last commit;
- **staged**: content is selected for the next commit;
- **clean**: no uncommitted change is detected.

Habit

Run `git status` before editing, before staging, before committing, before pulling, and before pushing. It provides context, not just success or failure.

Create a Small, Safe Change

Practise the workflow with student-owned documentation

Project Exercise

```
cd ~/quard_um6p_intership
mkdir -p student_work
echo "Git workflow observations" > student_work/git_notes.txt
git status
```

Engineering Intention

This creates a visible, low-risk change without modifying flight logic, simulation models, or shared configuration.

Repository Policy

If the project defines a specific student folder, naming convention, or contribution process, follow that documentation instead of inventing a parallel structure.

Inspect Changes with `git diff`

Review content before selecting it for a commit

Purpose and Syntax

```
git diff              # unstaged tracked changes
git diff --staged     # changes selected for commit
```

Project Workflow

```
echo "Repository inspected from the project root." \
  >> student_work/git_notes.txt
git diff
git status
```

Important Detail

An untracked file may not appear in ordinary `git diff`. Use `git status` to discover it, then stage it before reviewing with `git diff -staged`.

Stage Only the Intended Change

Build the next commit deliberately

Purpose and Syntax

```
git add <file-or-directory>
```

Preferred Beginner Workflow

```
git add student_work/git_notes.txt
git status
git diff --staged
```

Why Not Always Use `git add .`?

Staging everything may include logs, generated files, credentials, unrelated edits, or large assets. Add explicit paths until you can explain every staged change.

Create a Meaningful Commit

Record one coherent engineering change with a useful message

Purpose and Syntax

```
git commit -m "concise description of the change"
```

Useful Message

```
git commit -m \
  "docs: add student Git workflow notes"
```

It states the area and result.

Weak Messages

`update`, `changes`, `final`, and `fix stuff` do not explain engineering intent.

After Committing

Run `git status` and `git log -oneline -5`. The commit exists locally until it is pushed.

Read the Project History

Commits form an engineering narrative, not merely a backup list

Purpose and Useful Views

```
git log
git log --oneline
git log --oneline --graph --decorate --all
```

Questions the History Can Answer

- Which change introduced a launch instruction?
- When was a model or configuration updated?
- Which branch contains a student's contribution?
- What commit preceded a newly observed problem?

Branches Isolate Lines of Work

Experiment without immediately changing the shared main line

Inspect and Create

```
git branch
git switch -c docs/student-git-notes
```

`git branch` lists branches. `git switch -c` creates and enters a new branch.

Move Between Existing Branches

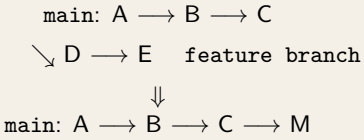
```
git switch main
git switch docs/student-git-notes
```

Before Switching

Check `git status`. Uncommitted changes may follow you, conflict with the destination, or prevent the switch. Branch names should describe the task, not the contributor's mood.

What Does Merge Mean?

Combine compatible histories after review and verification



Intuition

A merge combines changes from different lines of development. If both lines modify incompatible parts of the same content, Git asks an engineer to resolve the conflict.

Responsibility

A conflict is not Git failing. It is Git refusing to guess which engineering intention is correct. Resolve it by understanding both changes, then test the combined result.

Inspect the Remote Connection

Know where pulls come from and where pushes go

Purpose and Syntax

```
git remote -v
```

Expected Interpretation

`origin` is the conventional name created by cloning. The output normally shows fetch and push URLs for the GitHub repository.

Wrong-Remote Prevention

Before your first push, confirm that the owner and repository name are correct and that you have authorized contribution access. Do not replace `origin` randomly to bypass a permission problem.

Synchronize Before Publishing

Bring in remote work before sending your own commits

Safe Beginner Sequence

```
git status
git branch
git pull origin main
```

Meaning

`git pull` retrieves remote changes and integrates them into the current local branch. Read the output to determine whether the branch was already current, fast-forwarded, or requires conflict resolution.

Before Pulling

Start from a clean, understood working state. If local edits are unfinished, commit them appropriately or follow the instructor's recovery procedure rather than forcing the pull.

Publish Local Commits with `git push`

Synchronize verified history to GitHub

Purpose and Syntax

```
git push <remote> <branch>
```

Examples

```
git push origin main
# For a new contribution branch:
git push -u origin docs/student-git-notes
```

Rejected Push

A non-fast-forward rejection usually means the remote contains commits missing locally. Do not force-push. Retrieve and understand upstream changes, resolve any conflict, test, and then push normally.

The Everyday Internship Workflow

Observe, synchronize, modify, review, record, and share

1. **Enter and inspect:** move to the repository, then run `git status` and `git branch`.
2. **Synchronize:** pull the approved branch before beginning new work.
3. **Modify:** make one focused code, configuration, test, or documentation change.
4. **Review:** inspect `git status` and `git diff`.
5. **Stage:** add only the files intended for the next checkpoint.
6. **Verify staged content:** inspect `git diff -staged`.
7. **Commit:** use a concise message that explains the result.
8. **Synchronize again:** pull if necessary, resolve carefully, test, and push.
9. **Confirm on GitHub:** verify the branch, commit, and documentation online.

Team Rule

The exact branch and review policy is defined by the internship repository. Follow the documented collaboration workflow rather than assuming every contributor pushes directly to `main`.

Protecting the Project: Forking

Creating your own workspace without impacting the main repository

The "Golden Copy" Rule

The supervisor's main GitHub repository is the **Golden Copy**. If 10 students push their experimental code directly into it at the same time, the project will instantly break.

The Solution: "Forking"

A **Fork** is a completely separate, personal copy of the repository that lives on *your* GitHub account.

- You can freely modify it.
- You can completely break it.
- It **never** affects the supervisor's original code.

Your Safe Setup Workflow

Instead of cloning the main project directly, you will:

1. **Fork (In Browser):** Go to the supervisor's GitHub page and click the "Fork" button to create your copy.
2. **Clone (In Terminal):** Run `git clone` using the URL of *your* new fork, not the supervisor's URL.
3. **Push (In Terminal):** When you type `git push`, your code goes safely to your personal cloud repository.

Use `.gitignore` Deliberately

Keep generated, local, and sensitive artifacts out of history

The Purpose

A `.gitignore` file contains patterns telling Git to ignore specific untracked files, preventing them from accidentally being committed and pushed to GitHub.

Important Boundaries

- **Too Late:** Ignoring a file does *not* remove it from history if it was already committed previously.
- **Security:** Never rely on `.gitignore` as your only protection for credentials. Never commit passwords.
- **Bloat:** Use it to prevent uploading massive ROS 2 logs or compiled build files that crash repositories.

Typical Robotics Patterns

```
# Python cache
__pycache__/
*.pyc
# Local ROS 2 workspaces
build/
install/
log/
# Local environments
.venv/
# Sensitive secrets
.env
*.pem
```

Never Commit Secrets

Repository history can outlive the working file

Do Not Commit

- passwords, access tokens, API keys, or private SSH keys;
- private certificates or machine-specific credentials;
- confidential datasets or personal information;
- environment files containing secret values.

If a Secret Is Exposed

Stop sharing it, notify the responsible instructor or repository maintainer, revoke or rotate the credential, and follow the approved history-cleaning procedure. Merely deleting the file in a later commit may leave the secret in earlier history.

Better Pattern

Commit a documented template such as `.env.example` with placeholder names, then keep real secret values outside version control.

Keep the Repository Engineering-Friendly

Version the source of truth, not every generated artifact

Good Candidates

- source code and launch files;
- human-maintained configuration;
- small test fixtures;
- README files and reports;
- scripts needed to reproduce outputs.

Review Before Adding

- build and cache directories;
- large videos, models, bags, or datasets;
- temporary logs and screenshots;
- machine-specific settings;
- files obtainable from documented sources.

Reason

Large binary files make cloning, reviewing, and history management harder. Use the repository's approved storage and release strategy for necessary large assets.

Recover: Staged the Wrong File

Correct the selection without destroying the working change

Situation

You staged a file that should not be part of the next commit.

```
git status
git restore --staged path/to/file
git status
```

Effect

The file is removed from the staging area, but its working-tree modification is preserved for later review.

Do Not Guess Recovery Commands

Always inspect `git status` before and after recovery. Commands involving `-hard`, forced pushes, or history rewriting are outside the beginner workflow unless supervised.

Recover: Discard an Unwanted Local Edit

Use only when the last committed version is truly the desired state

For One Tracked, Unstaged File

```
git diff -- path/to/file
git restore path/to/file
git status
```

Irreversible Working-Tree Loss

`git restore path/to/file` discards the selected uncommitted edit. Copy valuable experimental content elsewhere or commit it on an appropriate branch before restoring.

Safer Question

Do I want to remove the file from the next commit, or destroy the actual edit? These are different intentions and require different commands.

Recover: Push Rejected or Pull Conflicted

Preserve evidence and resolve the real synchronization problem

Push Rejected

```
git status
git branch
git remote -v
git pull origin main
```

Read the output, integrate approved remote work, test, and push again. Never use force as the automatic response.

Merge Conflict

1. run `git status`;
2. open each conflicted file;
3. understand both alternatives;
4. remove conflict markers after deciding;
5. test the integrated result;
6. stage and commit the resolution.

Escalation Rule

If the correct engineering intention is unclear, stop and ask the file owner or instructor. A syntactically resolved conflict can still create incorrect robot behavior.

Two Computers, One Shared Repository

At the Computer Where Work Was Completed

```
git status
git add <intended-files>
git commit -m "describe the completed change"
git pull origin main
git push origin main
```

At the Other Computer

```
cd ~/quard_um6p_intership
git status
git pull origin main
```

Precondition

Each computer must be on the intended repository and branch. Uncommitted local edits must be reviewed before pulling, or they may conflict with the incoming work.

Guided Workflow

```
cd ~
git clone https://github.com/abdtnourji/quard_um6p_intership.git
cd quard_um6p_intership

git status
git remote -v
git log --oneline -5

mkdir -p student_work
echo "Git practice completed" > student_work/git_practice.txt

git status
git add student_work/git_practice.txt
git diff --staged
git commit -m "docs: add Git practice evidence"
git pull origin main
git push origin main
```

References

Primary documentation for continued practice


Git Project, *Git Documentation*.
https://git-scm.com/doc


Git Project, *A Tutorial Introduction to Git*.
https://git-scm.com/docs/gittutorial


GitHub Docs, *Git Basics*.
https://docs.github.com/en/get-started/git-basics


GitHub Docs, *Cloning a Repository*.
https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository


GitHub Docs, *Pushing Commits to a Remote Repository*.
https://docs.github.com/en/get-started/using-git/pushing-commits-to-a-remote-repository

Transition to the Technical Foundations

One foundation completed, the next one begins

Internship Learning Roadmap